Multi-Task Learning for Fashion-MNIST Classification and Regression

November 3, 2025

1 Approach and Model Architecture

This project implements a multi-task learning (MTL) model to simultaneously perform two tasks on the Fashion-MNIST dataset:

1. **Classification:** A 10-class classification task to identify the type of clothing (e.g., "T-shirt/top", "Trouser", "Coat").
2. **Regression:** A regression task to predict the "ink density" of the item. This target was engineered by calculating the **mean pixel value** of the original, un-normalized 28x28 image tensor.

1.1 Model Architecture (`MultiTaskCNN`)

A **hard parameter sharing** architecture was used. This model consists of a single shared backbone that extracts features for both tasks, which then splits into two separate "heads" to produce the final predictions.

- **Shared CNN Backbone:** A stack of 3 convolutional blocks with increasing filter depth: `(32, 64, 128)`. Each block consists of:
  1. `Conv2d` (3x3, padding=1)
  2. `BatchNorm2d`
  3. `ReLU`
  4. `MaxPool2d` (2x2)
  5. `Dropout2d` (with p=0.1, 0.2, 0.3 respectively)
- **Shared Fully-Connected Layer:** The output of the final conv block (128 channels, 3x3) is flattened to a vector of 1152 features and passed through a `Linear` layer (to 256 units), followed by `BatchNorm1d`, `ReLU`, and `Dropout`.
- **Task-Specific Heads:** The 256-unit shared vector is fed into two separate heads:
  1. **Classification Head:** A `Linear(256, 128)` layer followed by `BatchNorm1d`, `ReLU`, `Dropout`, and a final `Linear(128, 10)` layer to produce class logits.
  2. **Regression Head:** A `Linear(256, 128)` layer followed by `BatchNorm1d`, `ReLU`, `Dropout`, and a final `Linear(128, 1)` layer to predict the "ink" value.

1.2 Loss Function

The model was trained by optimizing a weighted sum of the two task losses:

$$L_{\text{total}} = \lambda_1 \times L_{\text{classification}} + \lambda_2 \times L_{\text{regression}} \tag{1}$$

Where:

- $L_{\text{classification}}$: `CrossEntropyLoss`
- $L_{\text{regression}}$: `MSELoss`
- λ_1, λ_2: Hyperparameters to balance the trade-off.

2 Hyperparameters and Experiments

All experiments were trained for 20 epochs, saving the model weights that achieved the lowest **total validation loss**. Experiments were logged to Weights & Biases (wandb).

2.1 Experiment Configurations (13 Runs)

A series of 13 experiments were conducted, primarily varying the λ_1 (classification) and λ_2 (regression) weights, plus single-task baselines.

- **Baseline Config:**
- `name`: `exp1_lambda1-1.0_lambda2-1.0`
- `optimizer`: Adam, `lr`: 0.001
- `lambda_1`: 1.0, `lambda_2`: 1.0
- `dropout`: 0.3
- `batch_size`: 128
- `filters`: (32, 64, 128)

2.2 Experiment Runs

The 13 runs included:

- 11 Multi-task runs with varying λ weights (e.g., `exp7_lambda1-1.0_lambda2-0.1` for classification focus, `exp11_lambda1-1.0_lambda2-10.0` for regression focus).
- 1 Classification-only run (`exp12`).
- 1 Regression-only run (`exp13`).

3 Wandb Plots & Best Model Selection

The following plots show the validation metrics for the experimental runs.

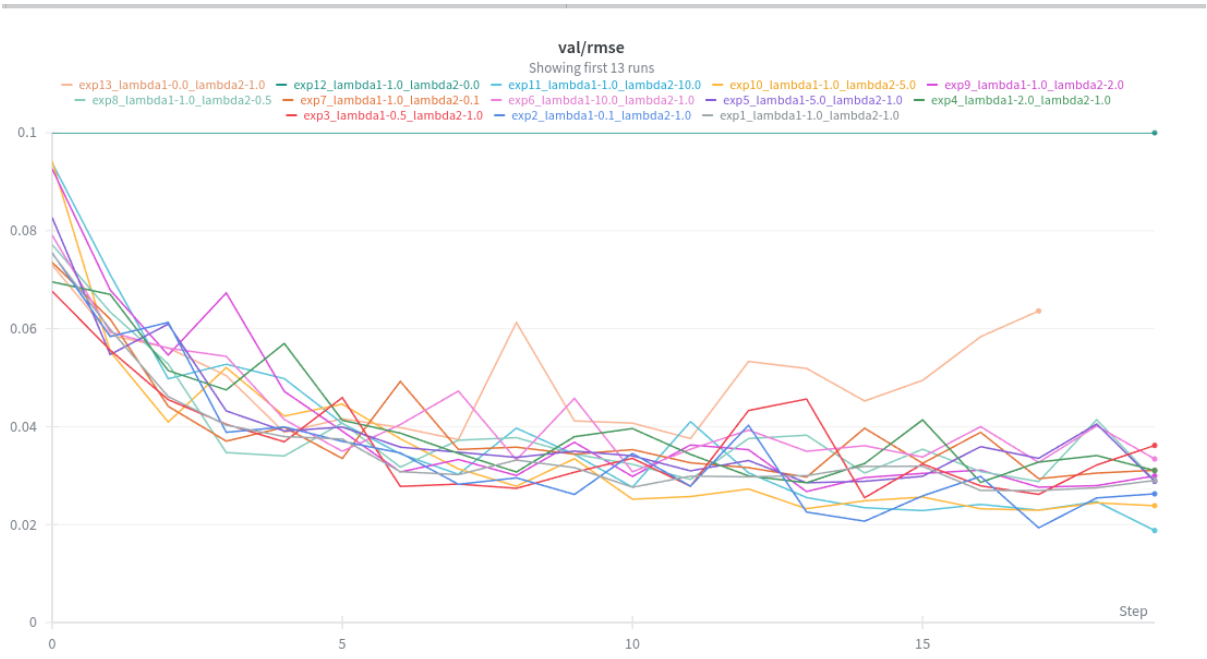


Figure 1: RMSE (validation) for the 13 experimental runs.

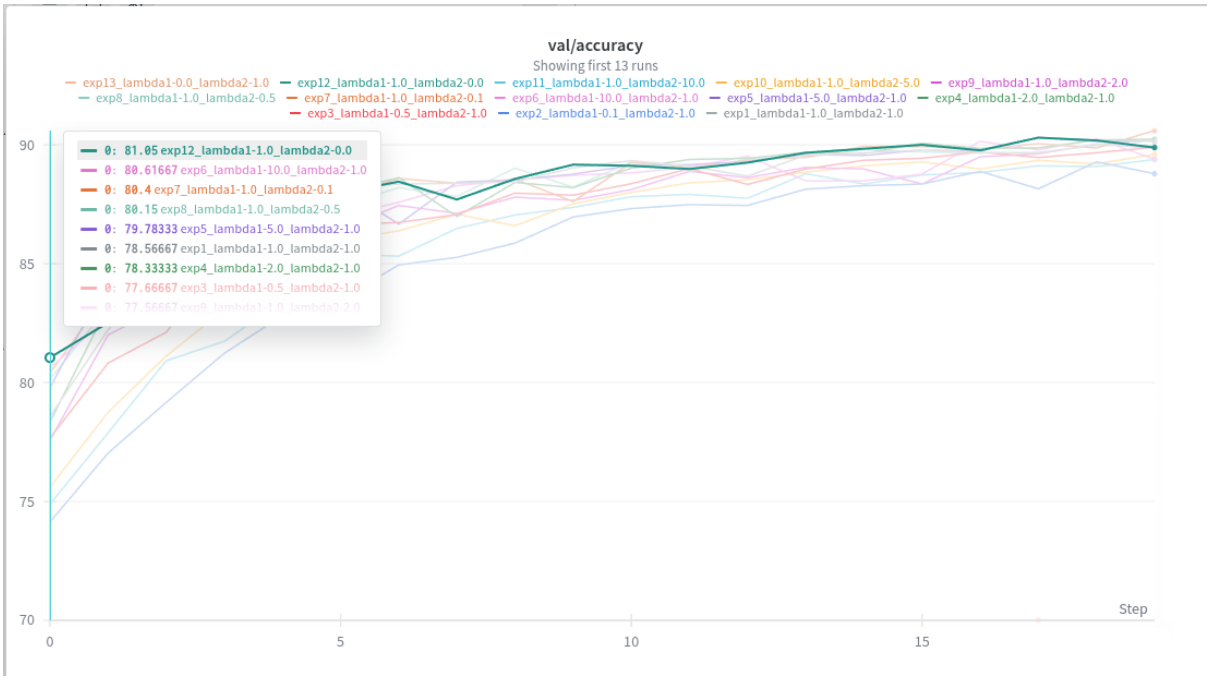


Figure 2: Accuracy (validation) for the 13 experimental runs.

3.1 Model Selection (Multi-Task Runs)

Based on the validation metrics from the multi-task experiments:

- **Model with Highest Validation Classification Accuracy:**
  - **Model:** `exp7_lambda1-1.0_lambda2-0.1`
  - **Peak Validation Accuracy: 90.60%**
  - **Corresponding Test Metrics:** Test Accuracy: **90.69%**, Test RMSE: **0.0305**

- **Model with Lowest Validation Regression RMSE:**
  - **Model:** `exp11_lambda1-1.0_lambda2-10.0`
  - **Lowest Validation RMSE: 0.0188**
  - **Corresponding Test Metrics:** Test Accuracy: **89.30%**, Test RMSE: **0.0185**

3.2 Validation Accuracy Trends

- The multi-task runs converged in the range **88.78%** – **90.60%** validation accuracy.
- The classification-only run (`exp12`) achieved a similar **90.62%**, while the regression-only run (`exp13`) failed classification (10.68%).

Key Insight: Classification performance was very strong across most multi-task runs, closely matching the performance of a dedicated classification-only model.

3.3 Validation RMSE Trends

- The multi-task RMSE values spanned **0.0188** – **0.0377**.
- The classification-only run (`exp12`) failed regression (Val RMSE: 0.8931).
- The regression-only run (`exp13`) achieved the lowest Val RMSE of 0.0187, slightly better than the best multi-task run (0.0188).

4 λ Trade-Off Analysis

The plots below show the results from the lambda-specific experiments.

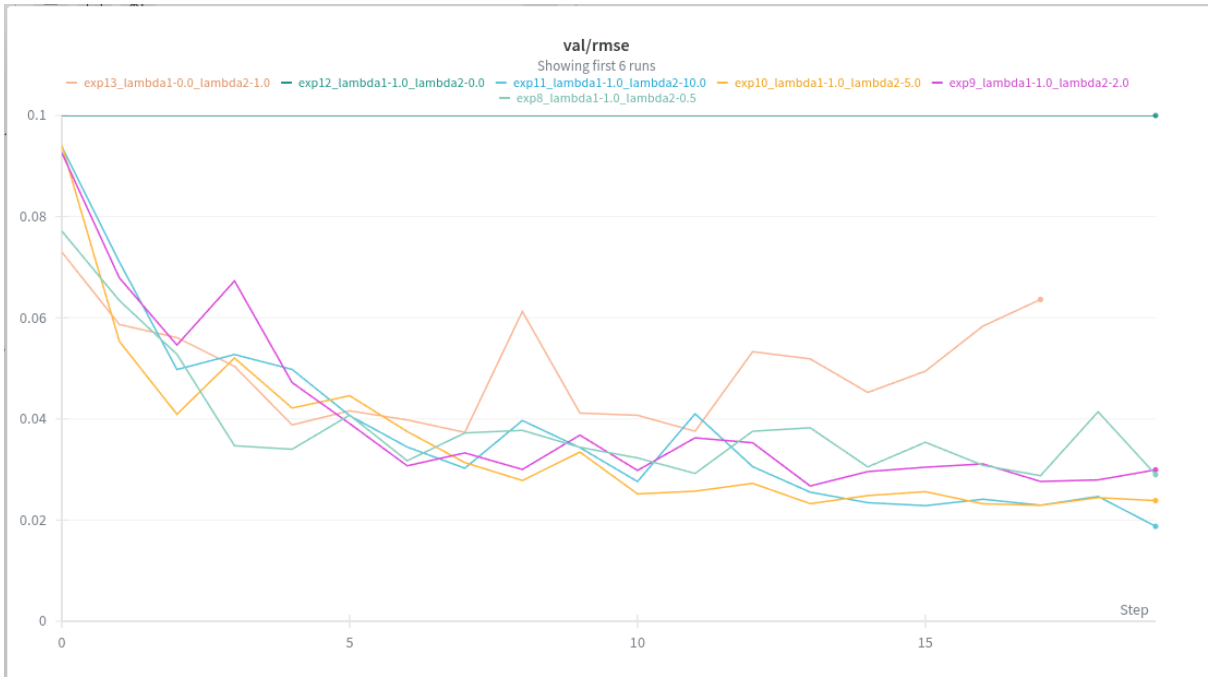


Figure 3: RMSE (validation) for the lambda-specific experiments.

These results clearly demonstrate the performance trade-off based on the test set evaluation of the best models:

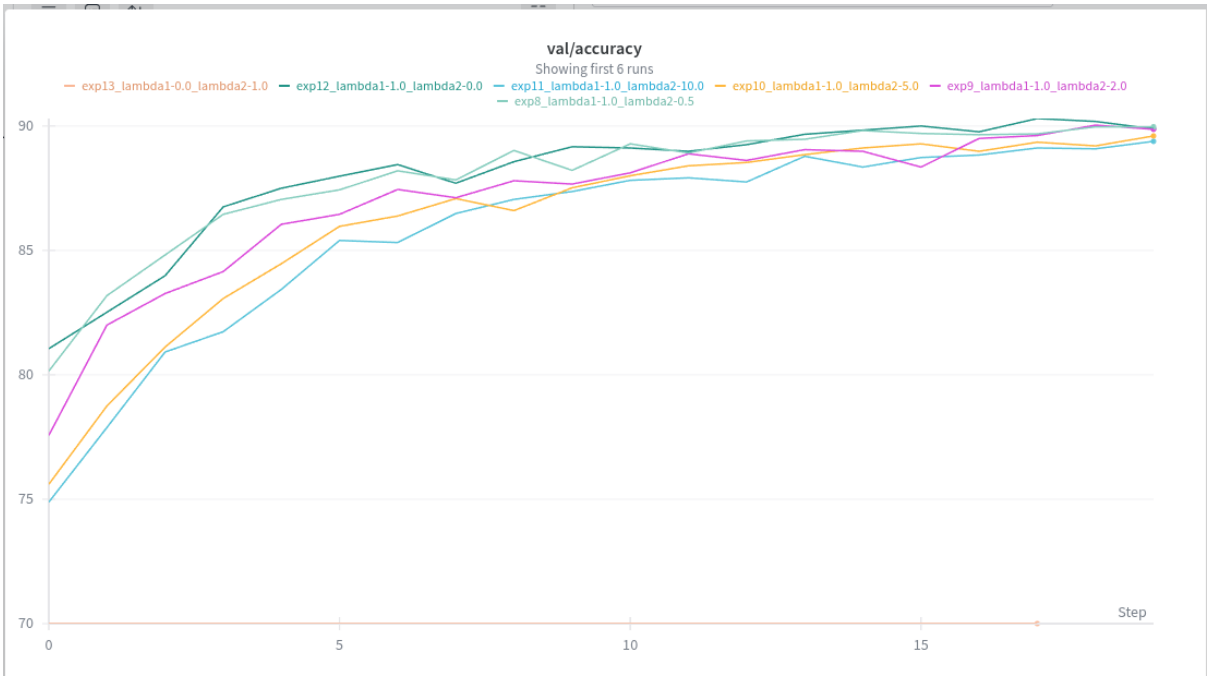


Figure 4: Accuracy (validation) for the lambda-specific experiments.

- **Prioritizing Classification (High λ_1, Low λ_2):** The `exp7_lambda1-1.0_lambda2-0.1` run achieved the **highest** `val/accuracy` (90.60%). This model generalized well, achieving a high **Test Accuracy of 90.69%** and a respectable **Test RMSE of 0.0305**.
- **Prioritizing Regression (Low λ_1, High λ_2):** Conversely, the `exp11_lambda1-1.0_lambda2-10.0` run achieved the **lowest** `val/rmse` (0.0188). This specialization yielded the best regression performance on the test set (**Test RMSE: 0.0185**) while maintaining a strong **Test Accuracy of 89.30%**.
- **Balanced Approach ($\lambda_1 = 1.0, \lambda_2 = 1.0$):** The `exp1_lambda1-1.0_lambda2-1.0` run provided a strong compromise, achieving a high Best Val Accuracy (**90.17%**) and a low Best Val RMSE (**0.0270**).

5 Feature Map Visualization & Interpretation

The `exp1_lambda1-1.0_lambda2-1.0` model was loaded to visualize the intermediate feature maps from its three convolutional blocks. This reveals a hierarchical feature extraction process.

- **Block 1 (32 channels, 14x14):** Acts as a basic feature extractor, activating on simple **edges and contours**. Different channels specialize in detecting horizontal, vertical, and diagonal lines.
- **Block 2 (64 channels, 7x7):** Combines simple edges into more complex **texture patterns** and local structures, crucial for distinguishing between items with similar shapes (e.g., "Shirt" vs. "Pullover").
- **Block 3 (128 channels, 3x3):** Captures **high-level, abstract features** and object parts, which are directly used by the classification head.

5.1 Multi-Task Synergy

The shared backbone works effectively because both tasks rely on the same underlying visual hierarchy:

- **Classification** uses the features to progress from edges → textures → object parts for identification.
- **Regression** uses these same features to estimate overall complexity and ink distribution. A high density of edge/texture activations suggests a higher mean pixel value, correlating directly with the "ink" target.